

# Lab – Short Summary



## Lab 1: Concurrency and Parallelism

- Running a program in different cores in a multi-core system
  - Task execution in each core is serial
  - Task execution in different cores in parallel
- Running a thread program in a single core
  - Execution in each thread is serial
  - Thread execution is concurrent

## Lab 2,3,4: Writing a multithread program in Java

- Manual
  - Extending Thread class
  - Implementing Runnable interface
  - What would be a potential problem?
- Using Executor Framework
- Using ForkJoin Framework

## Lab 5: Evaluating a parallel program

- Correctness
- Speedup, efficiency
- Memory usage

## Lab 5: When to use Parallel programming

- Time complexity
- Data size
- Whether or not the problem can be parallelized?

## Lab 5: What affects the speedup of a parallel program?

- Problem size
- Time complexity
  - $O(n)$ ,  $O(n^2)$ ,  $O(n^{\log n})$ ,  $O(n^3)$
- Your design
  - Work distribution
    - Task partition and data partition: load balancing
  - Data race problem
- Your program must produce the correct output

## Lab 6,7,8: Synchronization Mechanisms

- Objectives: solve data race problem in a multithread program
- Ideas
  - Create a critical section and ensure mutual exclusion in the critical section

## Lab 6,7,8: Synchronization Mechanisms

- **Objectives:** solve data race problem in a multithread program
- **Ideas**
  - Create a **critical section** and ensure **mutual exclusion** in the critical section



## Lab 6,7,8: Synchronization Mechanisms

### ➤ Monitor

- Use synchronized keyword to create critical sections
- Synchronized keyword used with
  - One or multiple methods
  - Block of code

### ➤ Lock

- Reentrant lock
- ReadWrite lock

## Lab 6,7,8: Synchronization Mechanisms

- Conditional variable
  - A Thread enter critical section when a given condition happen
- Semaphore
  - Control the number of threads which can enter the critical section at the same time

## Lab 6,7,8: Other approaches to protect a shared variable

- Atomic variable
  - A group of operations cannot be further divided
- Volatile keyword
  - Deal with cache coherence problem in a multi-core processor
  - The shared variable can be accessed only from/to the main memory

## Lab 6,7,8: Deadlock

- Happens when a synchronization mechanism is not used in a correct way
- Threads will never finish their execution
- Examples in lab: with Monitor and Lock

## Lab 6,7,8: Deadlock and Monitor

```
private static class Thread1 extends Thread {  
    public void run() {  
        synchronized (resource1) {  
            System.out.println("Thread 1: Holds resource R1...");  
  
            try {  
                TimeUnit.SECONDS.sleep(5);  
            } catch (InterruptedException e) {  
                // TODO Auto-generated catch block  
                e.printStackTrace();  
            }  
  
            System.out.println("Thread 1: Waits for resource R2...");  
  
            synchronized (resource2) {  
                System.out.println("Thread 1: Holds resource R1 and R2...");  
            }  
        }  
    }  
}
```

```
private static class Thread2 extends Thread {  
    public void run() {  
        synchronized (resource2) {  
            System.out.println("Thread 2: Holds resource R2...");  
  
            try {  
                TimeUnit.SECONDS.sleep(5);  
            } catch (InterruptedException e) {  
                // TODO Auto-generated catch block  
                e.printStackTrace();  
            }  
  
            System.out.println("Thread 2: Waits for resource R1...");  
  
            synchronized (resource1) {  
                System.out.println("Thread 2: Holds resource R1 and R2...");  
            }  
        }  
    }  
}
```



Never be executed

## Lab 6,7,8: Deadlock and Monitor

```
private static class Thread1 extends Thread {  
    public void run() {  
        synchronized (resource1) {  
            System.out.println("Thread 1: Holds resource R1...");  
  
            try {  
                TimeUnit.SECONDS.sleep(5);  
            } catch (InterruptedException e) {  
                // TODO Auto-generated catch block  
                e.printStackTrace();  
            }  
  
            System.out.println("Thread 1: Waits for resource R2...");  
  
            synchronized (resource2) {  
                System.out.println("Thread 1: Holds resource R1 and R2...");  
            }  
        }  
    }  
}
```

```
private static class Thread2 extends Thread {  
    public void run() {  
  
        synchronized (resource1) {  
            System.out.println("Thread 2: Holds resource R1 and R2...");  
        }  
  
        System.out.println("Thread 2: Waits for resource R1...");  
  
        System.out.println("Thread 2: Waits for resource R1...");  
  
        synchronized (resource2) {  
            System.out.println("Thread 2: Holds resource R2...");  
  
            try {  
                TimeUnit.SECONDS.sleep(5);  
            } catch (InterruptedException e) {  
                // TODO Auto-generated catch block  
                e.printStackTrace();  
            }  
        }  
    }  
}
```

## Lab 6,7,8: Deadlock and Lock

```
private static class Thread1 extends Thread {  
    public void run() {  
        synchronized (resource1) {  
            System.out.println("Thread 1: Holds resource R1...");  
  
            try {  
                TimeUnit.SECONDS.sleep(5);  
            } catch (InterruptedException e) {  
                // TODO Auto-generated catch block  
                e.printStackTrace();  
            }  
  
            System.out.println("Thread 1: Waits for resource R2...");  
  
            synchronized (resource2) {  
                System.out.println("Thread 1: Holds resource R1 and R2...");  
            }  
        }  
    }  
}
```

```
private static class Thread2 extends Thread {  
    public void run() {  
        synchronized (resource2) {  
            System.out.println("Thread 2: Holds resource R2...");  
  
            try {  
                TimeUnit.SECONDS.sleep(5);  
            } catch (InterruptedException e) {  
                // TODO Auto-generated catch block  
                e.printStackTrace();  
            }  
  
            System.out.println("Thread 2: Waits for resource R1...");  
  
            synchronized (resource1) {  
                System.out.println("Thread 2: Holds resource R1 and R2...");  
            }  
        }  
    }  
}
```



Never be executed

## Lab 9: Graph Search Algorithms

- Serial:
  - Understand how BFS and DFS work
- Parallel:
  - How to parallelize BFS and DFS (ideas)



## Lab 10: MPI Programming

- A program paradigm
- A Specification
- MPI is an API for communication between **processes**
- Deployed in both **shared-memory** and **distributed-memory** systems
- Different communication modes (P2P, collective)
  - Blocking/non-blocking send/receive operations (P2P)
  - Broadcast, Scatter, Gather

## Lab 11: Socket Programming

- Client - server model
- TCP (stream) and UDP (datagram):
  - Transport layer
  - Socket: IP address and port number

## Lab 11: Socket Programming

- Multicast
- Multicast is based on a **group membership** concept
  - Receivers have to join or subscribe to receive a packet sent to group
- The sender is not aware of each member in the group
- Based on UDP protocol

## Lab 12: Azure Lab

- Services provided by Azure: Virtual Machine with CPUs, RAM, SSD, OS
- Docker Containers
- Nested Virtualization

## Lab 13: Hadoop MapReduce

- Understand MapReduce programming paradigm
- Map function and Reduce function
- In the example in the lab, input and output are of WordCount.java are on the HDFS